

Non-Branching Is Not Enough: Output Purity as a Requirement for Promoting Observed Agent Behaviour to Deterministic Execution

Hem Chandra Padhalni

Independent Researcher

shramanpadhalni@gmail.com

September 12, 2026

Abstract

A recent class of systems reduces the cost of LLM-agent workflows by observing execution traces, identifying behaviour that has become repeatable, and promoting it to deterministic execution without a model in the loop. Published promotion criteria test *control-flow stability*: that repeated runs produce the same sequence of actions. We show this is necessary but not sufficient. A step can be perfectly non-branching and still fail to reproduce, because its *output* varies for reasons unrelated to its input — an embedded timestamp, an identifier, a process attribute. Promoting such a step yields an artifact that replays a stale value indefinitely, silently.

We report a counterexample drawn from 1,399 execution events captured from a live agent installation across 142 verified sessions. A faithful reimplementation of a published control-flow-only criterion promotes all five steps of a cloud governance workflow; one of those steps produces 31 distinct outputs across 31 traces for byte-identical input. We define output purity independently of control-flow stability, give an empirical oracle and a cheap single-trace detector, and measure both. We also report a contrast that was not anticipated: a corpus built from purpose-written tools exhibited no impurity at all, while one built from the platform’s general-purpose tools did — which is the kind these systems actually compile.

Our hypotheses and analysis plan were registered before data collection. The predicted impurity rate (20%) was not met (measured 6.2%, 95% CI [1.1%, 28.3%]); we report this as registered. The existence claim is confirmed. All artifacts, the corpus, and a single-command reproduction are public.

We prove repeatability, not correctness. Nothing here establishes that a promoted workflow is right — only that it reproduces.

1 Introduction

Agent workspaces increasingly observe their own use. Repeated work is synthesised into named, reusable units — skills, playbooks, recipes — which are then re-executed through the model on every invocation. The unit is a prompt, not a program: the two-hundredth identical run costs what the first did.

A recent line of work closes that gap by treating agent exploration as a *discovery* mechanism rather than a permanent execution model. Malik’s *Progressive Crystallization* [1] extracts successful behaviour from execution traces and promotes it through an execution-type taxonomy, from agent-orchestrated through hybrid to fully deterministic, on accumulated evidence. Wang et al.’s LOOP Skill Engine [2] records a tool trajectory on first run, extracts a parameterised branch-free template, and replays it with the LLM bypassed. Both report substantial savings: Malik more than 70% cost reduction in production with deterministic execution rising from zero to roughly 45% over eight months; LOOP 93.3–99.98% token reduction.

These systems must decide when observed behaviour has become safe to promote. The published criteria are properties of control flow. Malik’s Type 3 to Type 2 promotion requires “ ≥ 10 successful runs” and “ $\geq 90\%$ of runs produce the same action sequence” [1, Table II]. LOOP’s templates are branch-free by construction.

The gap. Control-flow stability does not imply reproducibility. Two runs may issue an identical sequence of operations with identical inputs and still produce different results, because a step’s output depends on something other than

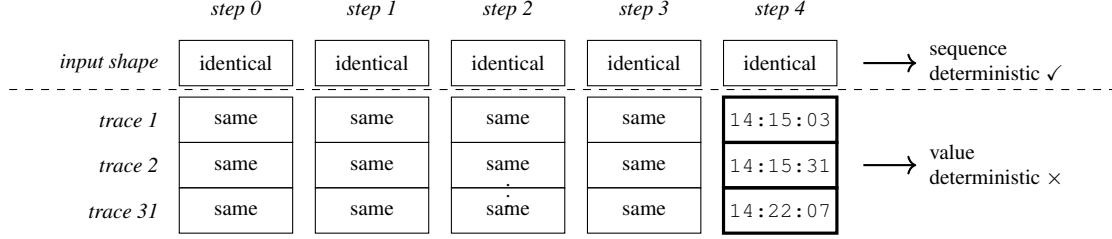


Figure 1: The gap, from the corpus. All five steps of the governance workflow share an identical input shape at every position across 31 traces, so the published control-flow criterion promotes the workflow. Step 4 appends to an audit log and returns a different timestamp on every run for byte-identical input. Promoting it yields an artifact that replays a stale value silently.

its input. We first encountered this in the first trace we ever captured: a directory listing whose output embedded a modification time.

```
-rw-r--r-- 1 1000 1000 99 Sep 09 14:15 ../quench-test.txt
```

The step is non-branching. Its input signature is stable. Its output differs on every execution. A control-flow-only criterion promotes it, and the resulting artifact replays a frozen timestamp for as long as it is trusted.

Contributions.

1. We distinguish *sequence determinism* from *value determinism* and argue the latter is a separate, necessary promotion criterion (§2).
2. We reimplement a published control-flow-only criterion and demonstrate, on real traces, that it promotes a step that does not reproduce (§5).
3. We give an empirical purity oracle and a cheap single-trace detector, and measure both, including a disagreement that we argue reflects a limit of the oracle rather than an error of the detector (§3, §6).
4. We report a corpus contrast: purpose-written tools exhibited no impurity; the platform’s general-purpose tools did (§7).
5. We implement the criterion in QUENCH, a system that compiles verified behaviour into signed, portable artifacts, and demonstrate execution with zero model calls and correct refusal on drift (§8).

Preregistration. Hypotheses, detector patterns, corpus rules, exclusion criteria and the analysis plan were committed publicly before data collection, with a timestamped commit. One deviation was recorded, also before the corrected analysis ran (§4). Our primary rate prediction was wrong; we report it as registered rather than adjusting it.

2 The Gap

Let a *workflow* be a finite ordered sequence of operations $s_1, \dots, s_n$, where each s_i invokes a tool with an input x_i and yields an output y_i .

Structural signature. Let $\sigma(x)$ denote the recursive shape of x with leaf values replaced by their types, dictionary keys sorted, and lists reduced to their element shape. Two inputs share a signature when they differ only in values.

Definition 1 (Sequence determinism). *A workflow is sequence deterministic over traces $T_1, \dots, T_m$ if every trace has the same length n and, for each position i , $\sigma(x_i^{(j)})$ is identical across all j .*

This is what published criteria test. Malik’s $\geq 90\%$ agreement is a relaxed form of it; LOOP’s branch-free templates are a strict form.

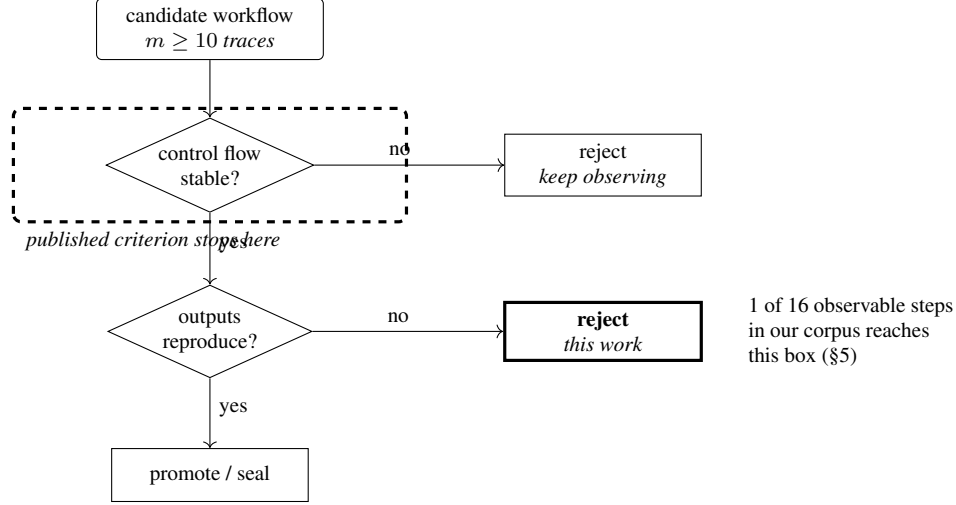


Figure 2: Two gates. Published promotion criteria test the first. A step that passes control-flow stability and fails output reproducibility is promoted by those criteria and rejected here; the bold box is the difference this paper measures.

Definition 2 (Value determinism, or purity). *A position i is pure over $T_1, \dots, T_m$ if, for every pair of traces j, k with $x_i^{(j)} = x_i^{(k)}$ (equal as values, not merely as shapes), we have $y_i^{(j)} = y_i^{(k)}$.*

The antecedent here is exactly what Khatchadourian calls *signature determinism* — “the fraction of runs with identical tool sequences *and* arguments” [3]. That work measures how often an agent reaches this regime. Purity asks what happens inside it, which that framework explicitly does not validate (§10).

The quantifier matters and is easy to get wrong. Purity is a claim about traces sharing identical input *values*. Comparing outputs across traces that merely share an input *shape* measures the diversity of the corpus, not the purity of the step — a mistake we made and corrected under preregistration (§4).

Proposition 1. *Sequence determinism does not imply value determinism.*

Witness: a step that reads the current time, appends to a log, or returns a directory listing carrying modification times. Its control flow never varies; its output always does. We exhibit such a step from real traces in §5.

Why it matters. A promoted artifact is executed *in place of* the agent. If a step is impure, the artifact returns a value that was correct when observed and is not correct now. Nothing fails; no error is raised; the audit trail records a successful execution. This is the worst failure shape available: silent, plausible, and persistent.

3 Purity: Oracle and Detector

We separate ground truth from what a running system can afford.

Oracle. Given m traces of one workflow, group positions by identical serialised input value and check whether serialised outputs agree. The oracle is exact with respect to the observed traces and requires a corpus.

Positions whose inputs never recur are *unobservable*: with a single occurrence there is nothing to compare. We exclude them from the denominator rather than assume purity.

Detector. A production system sees one output and must judge it. Detector v1 is seven regular expressions over the serialised output, frozen at preregistration: ISO 8601 timestamps, `ls`-style modification times, UUIDs, process identifiers, long hexadecimal runs, epoch seconds, and durations adjacent to `took/elapsed/duration`.

The detector is deliberately crude. Its purpose is not to match the oracle but to be affordable, and the interesting question — how far apart they are, and in which direction — is empirical.

4 Method

Preregistration. Before collecting data we committed: three hypotheses with a point prediction and interval; the seven detector patterns; corpus size and composition; inclusion and exclusion rules; the baseline reimplementation; and the analysis plan with its interval estimator. The commit is public and timestamped.

Deviation. One deviation was recorded, before the corrected analysis ran. The registered oracle definition compared outputs across traces sharing an input *shape*. Because different inputs legitimately produce different outputs, this classified a pure document-reading step as impure and yielded a pooled rate of 81.8% — an artifact of the definition. We corrected to comparison within identical input *values* and excluded unobservable positions. The point prediction, the interval, the falsification thresholds, the detector patterns and the baseline were left untouched. The flawed figure is preserved in the deviation record.

Corpus. 142 verified sessions producing 1,399 events across four workflows with ≥ 10 traces each, captured from Kiro Crew 0.5.0 / kiro-cli 2.21.2 on Ubuntu 24.04 via a `postToolUse` hook delivering full `tool_name`, `tool_input` and `tool_response` grouped by session. All data is synthetic and committed.

Session verification. Every session was checked against ground truth before inclusion: the trace file must have grown, and the artefact the workflow wrote must match the source document. This is not routine hygiene. During development a single unknown key in an agent specification caused the runtime to reject the entire specification and *silently substitute its default agent*. Four consecutive runs produced confident, plausible, entirely fabricated results with exit code zero — including invented lease terms and monetary values that appear in no document. A system that compiles observed behaviour must distinguish observation from narration, and narration is convincing. We return to this in §9.

Baseline. We reimplemented the control-flow-only criterion as published: ≥ 10 traces, identical step count, $\geq 90\%$ per-position signature agreement. Written before any purity statistic was computed, per the preregistration. Two ambiguities and our readings: we interpret “action” as one operation and compare input *shapes* (comparing values would make the criterion unsatisfiable, since no two runs share a document identifier); and we could not reproduce the “zero safety violations” condition outside its original environment, so we treat all traces as violation-free — which makes our baseline *more* permissive than published.

5 Results

5.1 H3: the existence claim (confirmed)

The cloud governance workflow — list a resource directory, read a resource configuration, read the policy, evaluate compliance, append the finding — passes the control-flow-only criterion at every position.

```
workflow: read -> read -> read -> shell -> shell    (31 traces)
baseline: PROMOTE - all positions >= 90% agreement

[0] pure    read    repeats=1   outputs=1   detector=ls_mtime
[1] pure    read    repeats=10  outputs=10  detector=-
[2] pure    read    repeats=1   outputs=1   detector=-
[3] pure    shell   repeats=10  outputs=10  detector=-
[4] IMPURE  shell   repeats=10  outputs=31  detector=iso_timestamp
```

Step 4 appends the finding to an audit log. Audit entries carry timestamps because that is what audit entries are; no impurity was contrived. Across 31 traces with byte-identical input it produced 31 distinct outputs.

The published criterion promotes this step. Purity rejects it.

Corpus	Observable	Impure	Rate
lease-abstraction	11	0	0%
cloud-governance-check	5	1	20%
Pooled	16	1	6.2%

Table 1: Impurity among observable steps. 95% Wilson CI [1.1%, 28.3%]. Registered prediction: 20%, interval 8–40%, falsified below 5%.

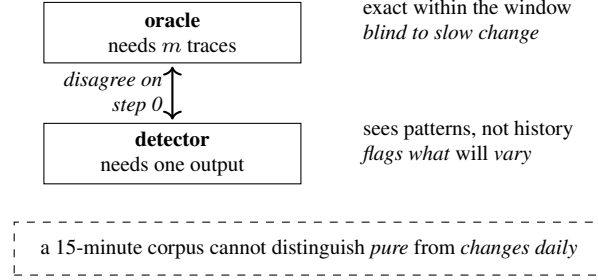


Figure 3: The two purity signals are complementary rather than ranked. The oracle is exact over observed traces and blind beyond them; the detector encodes knowledge the observation window cannot supply. Our single false positive (§6) is this disagreement, not an error.

5.2 H1: the impurity rate (below the registered interval)

The measured rate is above the falsification floor and below the predicted interval. Our prediction was too high. We report it as registered and do not claim the rate generalises: one impure step over sixteen observable positions cannot pin down a population parameter, and the interval says so.

5.3 H2: detector against oracle (below target)

	TP	FP	TN	FN	Precision	Recall
Detector v1	1	1	14	0	0.50	1.00

Table 2: Registered targets: precision ≥ 0.80 , recall ≥ 0.70 .

Recall is perfect. Precision is halved by a single false positive, which we argue below is not straightforwardly an error.

6 The False Positive

Step 0 is a directory listing. The detector flagged an `ls`-style modification time; the oracle scored the step pure.

Both are defensible. The pattern is genuinely present, and the listing *will* return different bytes once any file in that directory changes. But the directory did not change during a fifteen-minute corpus, so outputs were byte-identical and the oracle observed purity.

The oracle is ground truth only within the observation window. A step whose output changes daily is indistinguishable from a pure step in a corpus collected over minutes.

This is a limitation of every observation-first promotion system, including those we compare against, and it inverts the usual reading of a precision failure: the detector encodes knowledge the observation window cannot supply. We therefore treat the two signals as complementary rather than ranking one against the other, and recommend that promotion require *both* an observed purity result and a clean detector pass.

7 The Corpus Contrast

Our first corpus measured zero impurity across eleven observable steps. The cause was our own design: its tools were written to be deterministic — no clock, no randomness, no network, sorted keys on serialisation — with a self-test asserting byte-stable output. A corpus built from tools that cannot be impure cannot measure impurity.

We record this rather than quietly replacing it, because the second corpus makes the point sharper than either alone:

Purpose-written tools are pure because someone chose to make them so. General-purpose platform tools are not — and observation-first systems compile the second kind.

This bears directly on the systems under discussion. Malik’s system crystallises workflows built from operational tooling in a production cloud network environment; LOOP records trajectories over whatever tools an agent has been given. Neither operates over hand-written pure functions. We expect impurity to be *more* prevalent in those settings than in ours, not less — though we cannot demonstrate that from our data, and do not claim it.

8 Implementation

We implement the criterion in QUENCH, which compiles observed behaviour into signed, portable artifacts. Three properties are relevant here.

Verification is a gate, not a report. Branching and purity are checked independently and both must pass. On our corpus, three workflows seal and one is refused:

```
CANDIDATE [3]  read -> read -> read -> shell -> shell
REFUSED - not sealed
  purity: step 4 produced 31 distinct outputs for
          identical input across 31 traces
(passed: no_branching)
```

Artifacts are portable and signed. A sealed workflow is a file carrying its steps, their input shapes, inferred data-flow edges, and an Ed25519 signature over a provenance record: what verified it, against how many traces, under which strategies, when. To our knowledge no system in this class produces a transferable, independently verifiable artifact; existing implementations retain compiled behaviour inside the system that produced it.

Data-flow edges are inferred, not declared. If a step’s input value is byte-identical to a value reachable inside an earlier step’s output, by the same path, in every trace, that is an edge the agent was already following. On our corpus this recovers the full dependency structure of a six-step extraction workflow. Executed without a model:

```
Mill executed 6 steps in 478ms, 0 model calls
[1] extract_clause {"clause_type":"term","value":24}
[2] extract_clause {"clause_type":"annual_rent","value":521000}
[3] extract_clause {"clause_type":"escalation","value":2.5}
[4] extract_clause {"clause_type":"renewal","value":"NO"}
```

Values match the source document exactly. The agent path for the same work takes roughly 27 seconds and six model turns. Execution declines and defers to the agent on input-shape drift, signature failure, or revocation.

9 Threats to Validity

Sample size. One impure step. H3 is an existence claim and one instance suffices; H1 is a rate and this corpus cannot establish one.

Synthetic, self-authored workflows. Two workflow families, both written by us. The contrast in §7 is suggestive, not controlled.

Observation window. Fifteen minutes (§6). Slow-changing impurity is invisible to our oracle and likely undercounted.

Charitable baseline. Our reimplement is more permissive than published (§4). A stricter reading might reject the workflow for other reasons — though not, we note, for the reason purity rejects it.

Narration versus observation. During development, four runs produced fabricated results with exit code zero after a schema error silently substituted a different agent. Any system compiling observed behaviour is exposed to this: if the agent under observation is not the agent you believe it is, the corpus is invalid and nothing in the transcript reveals it. We mitigate by verifying every session against ground truth. We are not aware of published promotion criteria that address it.

Repeatability, not correctness. QUENCH compiles what an agent does. A consistently wrong workflow seals cleanly and becomes cheaper and faster to be wrong. This is a property of the whole approach, not of our implementation.

10 Related Work

Observation-first promotion. Malik [1] is the closest work and the direct subject of our comparison; the framing of agent exploration as discovery rather than permanent execution is his, as is the evidence-based promotion ladder with automatic demotion. Our contribution is a criterion his promotion test does not include. LOOP [2] reaches similar savings through one-shot recording and branch-free template replay; being single-shot, it has no opportunity to observe output variation at all, which makes the purity question more acute rather than less.

Measuring agent determinism. Khatchadourian’s DFAH [3] is the nearest work on the measurement side, and its decomposition of determinism is the one we build on. It defines *action determinism* as “the fraction of runs with identical tool sequences”, *signature determinism* as “the fraction of runs with identical tool sequences *and* arguments”, and *decision determinism* as “the fraction of runs with identical final decisions”, and reports that determinism and accuracy do not correlate across 4,700 runs — so both must be measured independently.

Signature determinism is precisely the precondition under which our question becomes askable: it is the regime in which the same tool is called with the same arguments. DFAH measures how often an agent reaches that regime; it does not ask what happens inside it. The framework “does not validate whether identical tool inputs produce identical tool outputs”, and its treatment of non-determinism concerns model seeding rather than values embedded in tool responses.

The division is clean and worth stating plainly. **DFAH measures whether the agent repeats. We measure whether the step reproduces, given that it does.** An agent may be perfectly signature-deterministic over a step that returns a fresh timestamp on every call; that is a success by DFAH’s metric and a promotion hazard by ours. That two independent lines of work arrived at the need to decompose determinism is, we think, evidence the decomposition is real.

Silent failure at the action boundary. Reddy et al. [4] report a different silent failure in tool-using agents: “the final database is wrong, but no tool ever raises an error”, because a policy-permissive tool executes a well-formed call whose state transition the domain forbids. They find 78% of failures in their benchmark are of this kind, and mitigate with deterministic read-only gates that inspect the proposed call and current state before a mutating call executes.

The mechanism is adjacent to ours and the boundary differs. Their gates fire before execution and ask *may this run?*; our check fires before promotion and asks *will this reproduce?* Both are silent failures — no error, no signal in the trace — which we read as evidence that the absence of an error is a poor proxy for correctness in this class of system, at more than one boundary.

Generation-first compilation. Compiled AI [5] constrains an LLM to generate business logic within validated templates, deployed as static code with no runtime inference. This is a different axis: the model is asked to write the workflow rather than observed performing it, so purity of observed steps does not arise.

Specification-first orchestration. Lobster, duckflux, and Conductor [6] execute deterministic workflows a human authored. Determinism is asserted by the author; our question does not apply.

Regression testing for agents. AgentAssay [7] addresses test efficiency for non-deterministic agent workflows. It is complementary: our oracle could be seen as a narrow, promotion-time instance of the same concern.

11 Conclusion

Promotion criteria for observed agent behaviour test control-flow stability. We show this is insufficient, exhibit a counterexample from real traces in which a published criterion promotes a step that does not reproduce, and give a checker that rejects it. We also report, from a mistake of our own, that purpose-written tools may show no impurity at all while general-purpose platform tools do — and that the second kind is what these systems compile.

Our registered rate prediction was wrong, and we report it unchanged. The existence claim stands. Both are reproducible from a single command against a public corpus.

Availability

Source, corpus, preregistration, deviation record and a one-command reproduction are available at <https://github.com/<user>/quench> under Apache-2.0. Reproduction requires no agent runs, API keys, or credentials.

References

- [1] A. Malik, “Progressive Crystallization: Turning Agent Exploration into Deterministic, Lower-Cost Workflows in Production,” arXiv:2607.07052, 2026.
- [2] X. Wang, K. Yu, X. Liang, L. Wang, and C. Han, “Good to Go: The LOOP Skill Engine That Hits 99% Success and Slashes Token Usage by 99% via One-Shot Recording and Deterministic Replay,” arXiv:2605.14237, 2026.
- [3] R. Khatchadourian, “Replayable Financial Agents: A Determinism-Faithfulness Assurance Harness for Tool-Using LLM Agents,” arXiv:2601.15322, 2026.
- [4] V. Reddy, S. R. Challaram, and A. Basu, “Reason Less, Verify More: Deterministic Gates Recover a Silent Policy-Violation Failure Mode in Tool-Using LLM Agents,” arXiv:2607.07405, 2026.
- [5] XY.AI Labs, “Compiled AI: Deterministic Code Generation for LLM-Based Workflow Automation,” arXiv:2604.05150, 2026.
- [6] Microsoft, “Conductor: Deterministic orchestration for multi-agent AI workflows,” 2026.
- [7] “AgentAssay: Token-Efficient Regression Testing for Non-Deterministic AI Agent Workflows,” arXiv:2603.02601, 2026.